

Implementation and Testing Documentation

1. Implementation Details

1.1 Project Structure

The project is built using Vite and organized modularly:

- * `/src/main.js`: The application entry point. Bootstraps the app, handles routing between views, and manages the global CR creation modal.
- * `/src/components/`: Contains UI component classes (Dashboard, Kanban, ImpactAnalysis, Traceability).
- * `/src/utils/`: Contains core logic utilities (`store.js` for state, `impactEngine.js` for graph traversal).
- * `/src/data/`: Contains initial seed data (`initialCRs.js`, `mockCodebase.js`) used for demonstration.
- * `/src/styles/`: Contains modular CSS (`variables.css`, `main.css`, `components.css`).

1.2 Key Implementation Patterns

- * **Vanilla SPA Routing:** The `App` class in `main.js` listens to `store` changes and swaps out the active UI component (Dashboard vs Kanban vs Impact) dynamically by managing the inner HTML of the main container.
- * **Drag and Drop API:** The `Kanban.js` component implements the native HTML5 Drag and Drop API, utilizing `dragstart`, `dragover`, and `drop` events to capture CR ID payloads and update state statuses.
- * **Impact Algorithm:** The `impactEngine.js` inverses the codebase dependencies to trace "who relies on what" rather than "what relies on who". It performs a BFS queue-based traversal to map direct and transitive impacts.

2. Testing Strategy

Since the application uses vanilla JavaScript, testing should cover business logic in isolation, followed by integration tests for the UI.

2.1 Unit Testing

Tools Recommended: **Vitest** or **Jest**.

State Store (`store.js`):

- * Test that `addChangeRequest` properly creates a CR with a generated ID and correctly appends dummy diffs.

- * Test that `updateCRStatus` alters the status of the correct CR without mutating others.

- * Test the Pub/Sub mechanism to ensure subscribers are called when state changes.

Impact Engine (`impactEngine.js`):

- * Mock a small component graph (e.g., A -> B -> C).

- * Assert that passing A as a seed component correctly flags B as direct and C as transitive.

- * Assert that risk scores accurately apply the decay formula based on distance, complexity, and churn rate.

2.2 Integration Testing

Tools Recommended: **Testing Library (DOM)** or **Cypress**.

- * **Component Rendering:** Ensure that updating `store.activeView` causes the `App` shell to mount the correct component.

- * **Form Submission:** Simulate filling out the CR Modal in `main.js`. Ensure submission calls `store.addChangeRequest` and the Kanban board reflects the new item.

2.3 Manual/End-to-End Testing (UAT)

Test cases for manual verification:

1. **CR Creation:** Click "Create CR", fill in the form, select multiple affected components, and submit. Verify it appears in the "Draft" Kanban column.

2. **Lifecycle Movement:** Drag a CR card from "Draft" to "In Progress". Drop it. Verify it visually anchors in the new column and its internal state is updated.

3. **Impact Simulation:** Click the Impact button on a CR card. Verify the view switches to Impact Analysis, and the correct risk scores and affected components are displayed based on the selected components during CR creation.

4. ****Code Traceability:**** Click the Diff button on a CR card. Verify the view switches to Traceability and the synthetic diffs are displayed.